TECHCATALYST DE 2026

# Intermediate Python & APIs

Error handling, files, classes , then your code reaches the internet: REST APIs with requests.

## Week 2
## Day 3

- Exceptions done right
- File I/O: CSV & JSON
- OOP for data models
- HTTP & REST
- The requests library

DAY 3

# Today's agenda

| Time | Block |
|---|---|
| 8:00 – 8:30 | Recap + drill review |
| 8:30 – 9:30 | Exceptions: try/except and failing loudly vs gracefully |
| 9:45 – 11:00 | File I/O: text, CSV, JSON — round-tripping data |
| 11:00 – 12:00 | OOP: classes as data models and connectors |
| 1:00 – 2:00 | HTTP & REST anatomy; instructor demo: live API |
| 2:00 – 4:30 | Lab: API ingestion exercises (3 levels) |
| 4:30 – 5:00 | Code reading circle + recap |

# Exceptions: plan for failure

```
try:
    fare = float(raw_value)
except ValueError:
    log_bad_row(raw_value)   # handle what you EXPECT
    fare = None
except Exception as e:
    print(f"unexpected: {e}") # catch-all LAST, and rarely
    raise                     # re-raise: don't swallow surprises
finally:
    close_connection()        # runs no matter what
```

**Pipeline rule:** catch specific exceptions you can handle; let everything else crash loudly. A silent except: pass corrupts data for weeks before anyone notices.

# Files: text, CSV, JSON

```
import csv, json

with open("zones.csv") as f:        # with 'closes it for you
    reader = csv.DictReader(f)      # each row = a dict!
    zones = [row for row in reader]

with open("trip.json") as f:
    trip = json.load(f)             # JSON -> dicts & lists

with open("out.json", "w") as f:
    json.dump(trips, f, indent=2)   # and back again
```

DictReader + json.load: yesterday's dict skills are exactly the file skills. Pandas does this at scale tomorrow.

02

# Objects, briefly

Just enough OOP to read real code and model your data.

# Classes: bundling data + behavior

```
class TripRecord:
    def __init__(self, vendor, fare, zone):
        self.vendor = vendor
        self.fare = fare
        self.zone = zone

    def is_valid(self):
        return self.fare is not None and 0 < self.fare <= 300

trip = TripRecord("CMT", 18.42, "JFK")
trip.is_valid()     # True
```

self = "this object". Mostly you'll USE classes others wrote , build one today so they stop being magic.

# Why DEs meet classes everywhere

### Data models

A record type with validation attached (you just wrote one). Libraries like Pydantic industrialize this.

### Connectors / clients

storage.Client(), requests.Session() , objects that hold config + connection state

### Frameworks

Airflow DAGs, Beam transforms, Spark sessions , you subclass or instantiate theirs

Goal today is reading fluency, not OOP mastery , composition and dunders can wait.

03

# Talking to APIs

Where most pipeline data actually comes from.

# HTTP in one slide

| Piece | Example | Meaning |
|---|---|---|
| Method | GET / POST | Read / create (DEs mostly GET) |
| URL + params | /v1/trips?zone=JFK&limit=100 | What you're asking for |
| Headers | Authorization: Bearer <key> | Who you are, what format you want |
| Status code | 200 / 404 / 429 / 500 | OK / not found / slow down! / their bug |
| Body | JSON (usually) | The data — straight into dicts |

429 Too Many Requests is the one that bites pipelines , respect rate limits, back off, retry.

# requests: the whole workflow

```
import requests

url = "https://data.cityofnewyork.us/resource/erm2-nwe9.json"
params = {"$limit":50, "$where":"borough = 'BROOKLYN'"}

resp = requests.get(url, params=params, timeout=10)
resp.raise_for_status()      # crash loudly on 4xx/5xx
records = resp.json()        # list of dicts

print(resp.status_code, len(records))
200 50
```

timeout always. raise_for_status always. These two lines separate professionals from tutorials.

# Pagination, rate limits, secrets

## Pagination

APIs cap results per call (limit/offset or page tokens)

Loop until empty page; accumulate

Lab level 2 makes you do this

## Rate limits

429 = back off

time.sleep() between calls; exponential backoff on retry

Be a polite client , your IP (and team) get banned together

## API keys

NEVER in code, NEVER in Git

Environment variables: os.environ["API_KEY"]

One leaked key = a real incident at work

# requests vs httpx

### requests

The classic, everywhere in tutorials

Sync only, no HTTP/2

Great for simple scripts

### httpx

Nearly identical API to requests

Sync and async in one library

HTTP/2, timeouts, full type hints

### Why httpx now

Migration is almost copy and paste

Async unlocks concurrency

Used by FastAPI and Starlette

# Sync vs async with httpx

```
# sync: each call waits for the one before it
with httpx.Client() as client:
    for url in urls:
        r = client.get(url, timeout=10)

# async: the calls overlap their waiting
async with httpx.AsyncClient() as client:
    coros = [client.get(u, timeout=10) for u in urls]
    responses = await asyncio.gather(*coros)
```

Async wins when you wait on the network, which is most of ingestion. It does not speed up CPU work.

# API vs SDK

## API

The HTTP contract: URL, method, headers, body

You build and send the request

Works from any language

## SDK

A language wrapper around the API

Handles auth, signing, retries, paging

boto3 for AWS, google-genai for Gemini

## When to use

SDK for production cloud work

Raw HTTP to learn or for tiny calls

Same result, the SDK is shorter and safer

# Hide your keys with .env

```
# .env  (never commit this file)
GOOGLE_API_KEY=your-key-here
AWS_ACCESS_KEY_ID=your-id
AWS_SECRET_ACCESS_KEY=your-secret

# .gitignore
.env

# load it in Python
from dotenv import load_dotenv
import os
load_dotenv()
key = os.environ["GOOGLE_API_KEY"]
```

uv add python-dotenv. Keys live in .env, .env lives in .gitignore, secrets never reach Git.

# APIs in the AI era

## AI is an API call

Claude, OpenAI, and Gemini are APIs

Call them directly or via an SDK

Today you called Gemini both ways

## So is everything else

AWS, GCP, weather, maps, payments

Your pipelines are made of API calls

This skill does not go away

## Even MCP

MCP lets AI agents use tools

Those tools are API calls underneath

Knowing requests is the durable skill

DEMO

# Demo: NYC Open Data, live

- **We'll pull NYC 311 service request records from the Socrata API together:**
  - Inspect the endpoint in the browser first , see raw JSON
  - GET with params; check status; .json() into dicts
  - Filter with $where; page with $limit/$offset
  - Write results to JSON and CSV files (this morning's skills)
- **Then it's your turn , same pattern, three difficulty levels, in the lab.**

GROUP ACTIVITY

# Lab , API ingestion, three levels

**Goal ,** write a real ingestion script by hand: call, validate, paginate, persist.

**150 min**

1. Level 1: GET 100 records from NYC Open Data; print count + 3 sample fields; save raw JSON.
2. Level 2: paginate to 1,000+ records; handle errors with try/except + raise_for_status; add polite sleep.
3. Level 3: clean records with your Day 2 functions (types! Nones!) and write a tidy CSV.
4. Stretch: wrap it in a SocrataClient class with get_page() and get_all() methods.

**Deliverable:** ingest_api.py + raw JSON + cleaned CSV, committed via a PR your partner reviews.

RECAP

# Key takeaways

1. Catch the exceptions you expect; let surprises crash loudly. Never except: pass.
2. with open(…) + csv/json modules round-trip data; dicts are the common currency.
3. Classes bundle data + behavior , you need reading fluency more than design skills.
4. requests + timeout + raise_for_status + pagination = a real ingestion script.
5. Tomorrow: pandas & Polars do this at scale , and your script ships data to GCS.